

Contactos Covid-19

Algoritmos e Estruturas de Dados
Enunciado do Trabalho Prático, versão 2.2 – 22-out-2020

Alterações no enunciado

- v. 2.2 - Alterações nas secções 3.3 e 3.4: esclarecimento do output dos comandos **IC** e **RC** quando há tentativa de inserção ou remoção de contacto de um utilizador consigo próprio. Esclarecimento sobre a ordem de verificação dos erros nestes comandos.
- v. 2.1 - Alterações nas secções 3.13 e 3.14: Introdução de mensagens nos comandos LMC e LMG para detectar quando o contacto e grupo não têm mensagens.
- v. 2.0 - Alterações nas secções 3.13 e 3.14: Correção de contradição da versão 1.0 nos comandos LMC e LMG e simplificação destes dois comandos.

Conteúdo

1	Introdução e notas importantes	3
2	Desenvolvimento da aplicação <i>Contactos COVID-19</i>	3
2.1	Descrição do problema	3
2.2	Conceitos e definições	3
2.3	Dados e resultados	4
2.4	Especificação do sistema	4
3	Comandos	5
3.1	Comando IU – inserir novo utilizador	6
3.2	Comando DU – consulta dados de utilizador	6
3.3	Comando IC – inserir contacto	6
3.4	Comando RC – remover contacto	7
3.5	Comando LC – listagem de contactos	8
3.6	Comando IG – inserir novo grupo	9
3.7	Comando DG – consulta dados de grupo	9
3.8	Comando RG – remover grupo	10
3.9	Comando IP – inserir participante num grupo	10
3.10	Comando RP – remover participante de um grupo	11
3.11	Comando LP – listagem de participantes	12
3.12	Comando IM – inserir mensagem	12
3.13	Comando LMC – listar mensagens de contacto	13
3.14	Comando LMG – listar mensagens de grupo	18
3.15	Comando FIM – terminar	19
4	Fases do trabalho e avaliação	20

5	Submissão	20
6	Requisitos do Mooshak	21
7	Testes e discussão	21

1 Introdução e notas importantes

Este documento descreve o trabalho prático da disciplina de Algoritmos e Estruturas de Dados do 2º ano do Mestrado Integrado em Engenharia Informática. Os alunos podem e devem tirar todas as dúvidas com os docentes da disciplina.

Trabalho em grupo de 2 alunos a desenvolver respeitando os princípios do *Código de Ética* do Departamento de Informática. Não são aceites grupos individuais sem autorização por escrito (por ex. por email) de um docente.

Trabalho com 2 fases de entrega. Ver detalhes sobre a diferença entre as fases e versões possíveis na secção 4. Ver as datas de entrega na secção 5.

2 Desenvolvimento da aplicação *Contactos COVID-19*

O objectivo deste trabalho é o desenvolvimento de um sistema de identificação de contactos para detecção de possível transmissão de Covid-19.

2.1 Descrição do problema

Pretende-se criar uma aplicação que facilite identificar os contactos dos seus utilizadores. Por exemplo, um aluno de AED do turno prático presencial P_x , poderá querer ser notificado se algum dos colegas desse turno estiver infectado com Covid-19. Para isso o aluno pode usar esta aplicação e aderir ao grupo de alunos desse turno.

Os utilizadores podem divulgar informação através de mensagens (por exemplo, um utilizador u pode querer publicar a mensagem “*Fiz o teste da Covid-19 e o resultado foi negativo :-)*”). Estas mensagens serão distribuídas pelos contactos do utilizador u e pelos grupos a que u pertence.

Esta aplicação tem como funcionalidades principais:

- registar um utilizador u ;
- registar um contacto u_c (pessoa com quem o utilizador u contacta – por exemplo, o contacto u_c pode ser um funcionário do café que o utilizador u costuma frequentar);
- registar um grupo de contactos (por exemplo, *P2-AED*, *Judo-Caparica-Quarta*);
- aderir a um grupo;
- consultar publicações (mensagens) de um contacto ou um grupo.

2.2 Conceitos e definições

Existe um conjunto de conceitos associados ao sistema a desenvolver e que se descreve a seguir:

Utilizador – Um utilizador é identificado por um login (uma cadeia de caracteres sem espaços).

Mensagem – Os utilizadores podem publicar e receber mensagens de texto.

Contacto – A cada utilizador estão associados os seus contactos, que são outros utilizadores registados no sistema. A relação de contacto é recíproca, ou seja, se o utilizador u_1 é contacto do utilizador u_2 , então o u_2 também é contacto do u_1 .

A partir do momento em dois utilizadores u_1 e u_2 se registam como contacto um do outro,

o u_1 vai receber todas as mensagens enviadas pelo u_2 e vice-versa. Se estes utilizadores removerem a relação de contacto, as mensagens já enviadas continuam a ser visíveis (ou seja, as mensagens que já foram enviadas pelo u_1 continuam a poder ser lidas pelo u_2 e vice-versa) mas se forem registadas novas mensagens, estas já não serão enviadas ao outro utilizador.

Grupo – Um grupo é identificado pelo seu nome (uma cadeia de caracteres sem espaços).

Participante – A cada grupo estão associados os seus participantes, que são utilizadores registados no sistema. Cada utilizador pode pertencer a 10 grupos no máximo.

Tal como acontece com a relação de contacto entre dois utilizadores, enquanto um utilizador for participante de um grupo, as suas mensagens são enviadas ao grupo. Caso o utilizador decida deixar de ser participante no grupo, as mensagens registadas anteriormente não são apagadas do grupo, mas o grupo não recebe as novas mensagens registadas pelo utilizador.

Uma mensagem é composta por um título, um texto e um URL. Quando um utilizador escreve uma mensagem, esta (1) é guardada na sua colecção de mensagens, (2) é enviada todos os seus contactos e (3) é também enviada a todos os grupos em que o utilizador é participante. As mensagens são guardadas por ordem de recepção, da mais recente para a mais antiga.

Os participantes de um grupo não recebem as mensagens de outros participantes do grupo (a não ser que também estejam registados como seus contactos), mas podem consultar as mensagens quando acedem ao grupo. Assim, a consulta das mensagens de um grupo só é permitida aos seus participantes.

2.3 Dados e resultados

Os campos *login* de utilizador e nome de grupo (*nomeGrupo*) são cadeias de 20 caracteres, sem espaços. Os nomes de utilizador (*nomeUtilizador*) têm 50 caracteres e todos os textos, assim como os URLs, têm 200 caracteres. As restantes cadeias de caracteres terão um comprimento de 20. As cadeias de caracteres de dados a inserir no sistema não deverão conter mudanças de linha (símbolo $\leftarrow$). A *idade* dos utilizadores é um inteiro.

A introdução de dados poderá ser feita em minúsculas ou em maiúsculas. Por exemplo, para a aplicação, o grupo com o nome *P10-AED* deverá ser igual a *P10-aed* e a *p10-aed*. O output dos dados contidos no sistema deverá ser sempre feito em maiúsculas.

Não existem restrições quanto ao número de utilizadores ou de grupos que podem existir. O número de contactos ou de mensagens associados a um utilizador também não é limitado superiormente mas, cada utilizador pode aderir, no máximo, a **10 grupos**. Também o número de participantes e de mensagens de um grupo não têm limites. Poderemos, no entanto, prever um número de 12 000 utilizadores e 4 000 grupos. Podemos também prever que cada utilizador gera 20 mensagens por mês.

2.4 Especificação do sistema

A interação do utilizador com o programa é feita sempre através de comandos, descritos na secção seguinte. O sistema deverá ler comandos da entrada padrão (System.in), processando-os um a um e enviando os resultados para a saída padrão (System.out). A execução do programa pode ser assegurada com a introdução de dados e respetiva apresentação de resultados em ficheiro, através do redireccionamento do input e do output.

3 Comandos

Nesta secção apresentam-se os vários comandos que o sistema deve ser capaz de interpretar e executar. Nos exemplos apresentados, diferenciamos o **texto escrito pelo utilizador** da retroacção escrita pelo programa na consola, ao executar o comando.

Vários comandos têm parâmetros. A entrada e a saída deverão respeitar o formato rígido que se indica neste documento. Pode assumir que o utilizador não cometerá erros na introdução de argumentos nos comandos, para além dos descritos neste enunciado, ou seja, apenas tem de tratar as situações de erro descritas aqui, pela ordem que são descritas.

Pode assumir que o utilizador apenas usa argumentos em número e de tipos correctos. No entanto, pode acontecer que os argumentos passados a um desses comandos tenham algum valor inválido no contexto do problema. Por esse motivo, teremos de testar esses argumentos exactamente pela ordem especificada neste enunciado.

Na leitura de comandos o interpretador não deve fazer distinção entre maiúsculas e minúsculas. Por exemplo, para o comando **IU** (comando usado para registar um novo utilizador) o interpretador deve aceitar como válidas as alternativas **IU**, **iu**, **Iu** e **iU**.

Nos vários exemplos que se seguem, o símbolo $\leftarrow$ denota a mudança de linha. Cada comando termina com duas mudanças de linha.

A tabela 1 apresenta os comandos do sistema, cujos detalhes são apresentados nas secções seguintes. Caso o utilizador introduza um comando inexistente, o programa ignora. Por exemplo o comando inexistente **desconhecido** teria este efeito:

```
> desconhecido↵
↵
>
```

Tabela 1: Comandos

Comando	Descrição	Detalhes
IU	inserção de novo utilizador	Secção 3.1
DU	informação sobre um utilizador	Secção 3.2
IC	inserção de contacto entre dois utilizadores	Secção 3.3
RC	remoção de contacto entre dois utilizadores	Secção 3.4
LC	listagem de contactos de um utilizador	Secção 3.5
IG	inserção de novo grupo	Secção 3.6
DG	informação sobre um grupo	Secção 3.7
RG	remoção de um grupo	Secção 3.8
IP	inserção de participante num grupo	Secção 3.9
RP	remoção de participante de um grupo	Secção 3.10
LP	listagem de participantes de um grupo	Secção 3.11
IM	inserção de uma mensagem	Secção 3.12
LMC	listagem de mensagens de um contacto	Secção 3.13
LMG	listagem de mensagens de um grupo	Secção 3.14
FIM	termina a execução do programa	Secção 3.15

3.1 Comando IU – inserir novo utilizador

Regista um novo utilizador no sistema. Para realizar um registo deve ser dado sempre o login, que tem que ser único. São também introduzidos o nome do utilizador, a sua idade, localidade e profissão. Este comando só tem sucesso se não existir um utilizador com o login dado.

O formato do comando é o seguinte (parâmetros com estilo **bold**):

```
> IU login nomeUtilizador↵
idade localidade↵
profissao↵
↵
Registo de utilizador executado.↵
>
```

Exemplo da execução do comando IU quando o login *aa* já existe no sistema. Neste caso, escreve-se na consola Utilizador ja existente.

```
> IU aa Antonio↵
23 Caparica↵
estudante↵
↵
Utilizador ja existente.↵
>
```

3.2 Comando DU – consulta dados de utilizador

Consulta os dados de um dado utilizador. Este comando só tem sucesso se o utilizador existir. Quando este comando é executado, a informação do utilizador é escrita na consola.

O formato geral do comando é o seguinte:

```
> DU login↵
↵
login nomeUtilizador idade
localidade profissao
>
```

Exemplo da execução do comando DU quando o utilizador com o login dado (ex. *bb*) não existe.

```
> DU bb↵
↵
Inexistencia do utilizador referido.
>
```

3.3 Comando IC – inserir contacto

Regista contacto entre dois utilizadores. A relação de contacto é recíproca, isto é, o comando IC **login1 login2** é equivalente ao comando IC **login2 login1**. Se o utilizador com o login *login2* pertence à colecção de contactos do utilizador com o login *login1*, então o utilizador com o login *login1* pertence à colecção de contactos do utilizador com o login *login2*.

O formato geral do comando é o seguinte:

```
> IC login1 login2↵
↵
Registo de contacto executado.↵
>
```

Este comando só tem sucesso se (1) os dois utilizadores existirem, (2) se *login1* é diferente de *login2* e se (3) ainda não tinha sido registado o contacto entre os dois utilizadores. A ordem de verificação de erros neste comando é: (1) existência dos utilizadores, (2) os utilizadores são diferentes, (3) existência do contacto entre os dois utilizadores.

Quando um dos utilizadores, ou ambos, não existe no sistema, escreve-se na consola **Inexistencia do utilizador referido**. Exemplo da execução do comando **IC** quando o utilizador com o login *bb* não existe no sistema.

```
> IC bb aa↵
↵
Inexistencia do utilizador referido.↵
>
```

Note-se que não se permite registar um contacto de um utilizador consigo próprio. Quando *login1* é igual a *login2* (e este utilizador existe no sistema) obtém-se o seguinte resultado:

```
> IC aa aa↵
↵
Existencia do contacto referido.↵
>
```

Por último verifica-se se os utilizadores já são contactos um do outro. Exemplo da execução do comando **IC** quando o contacto entre os utilizadores com os logins *aa* e *cc* já foi registado.

```
> IC aa cc↵
↵
Existencia do contacto referido.↵
>
```

3.4 Comando **RC** – remover contacto

Remove contacto entre dois utilizadores. Como a relação de contacto é recíproca, se o utilizador com o login *login2* é removido da colecção de contactos do utilizador com o login *login1*, então o utilizador com o login *login1* também é removido da colecção de contactos do utilizador com o login *login2*.

O formato geral do comando é o seguinte:

```
> RC login1 login2↵
↵
Remocao de contacto executada.↵
>
```

Este comando só tem sucesso se (1) os dois utilizadores existirem, (2) se *login1* é diferente de *login2* e se (3) o contacto entre os dois utilizadores já tinha sido registado. A ordem de

verificação de erros neste comando é: (1) existência dos utilizadores, (2) os utilizadores são diferentes e (3) existência do contacto entre os dois utilizadores.

Quando um dos utilizadores, ou ambos, não existe no sistema, escreve-se na consola **Inexistencia do utilizador referido**. Exemplo da execução do comando **RC** quando o utilizador com o login *bb* não existe no sistema.

```
> RC bb aa↵
↵
Inexistencia do utilizador referido.↵
>
```

Exemplo da execução do comando **RC** quando o *login1* e *login2* são iguais (e o utilizador com este login está registado no sistema).

```
> RC aa aa↵
↵
Contacto nao pode ser removido.↵
>
```

Exemplo da execução do comando **RC** quando o contacto entre os utilizadores com os logins *aa* e *cc* não foi registado, ou já foi removido.

```
> RC aa cc↵
↵
Inexistencia do contacto referido.↵
>
```

3.5 Comando **LC** – listagem de contactos

Lista os contactos de um utilizador. Listagem, por ordem lexicográfica do login, dos contactos de um dado utilizador. Para cada contacto, escreve-se na consola o seu login e nome.

O formato geral do comando é o seguinte:

```
> LC login↵
↵
loginC1 nomeC1↵
loginC2 nomeC2↵
...
loginCn nomeCn↵
>
```

loginCi e *nomeCi* representam o login e nome do *i*-ésimo contacto do utilizador com login *login*.

Este comando só tem sucesso se (1) o utilizador com login *login* existir, e se (2) este utilizador tem contactos registados. A ordem de verificação de erros neste comando é: (1) existência do utilizador e (2) existência de contactos deste utilizador.

Quando o utilizador não existe no sistema, escreve-se na consola **Inexistencia do utilizador referido**. Exemplo da execução do comando **LC** quando o utilizador com o login *bb* não existe no sistema.


```
> LC bb↵
↵
Inexistencia do utilizador referido.↵
>
```

Quando o utilizador não tem contactos, escreve-se na consola *Inexistencia de contactos*. Exemplo da execução do comando *LC* quando o utilizador *cc* não tem contactos registados no sistema.

```
> LC aa↵
↵
Inexistencia de contactos.↵
>
```

3.6 Comando IG – inserir novo grupo

Regista um novo grupo no sistema. Para criar um novo grupo deve ser dado o seu nome, que tem que ser único. É também introduzida uma descrição do grupo. Este comando só tem sucesso se não existir um grupo com o nome dado.

O formato geral do comando é o seguinte:

```
> IG nomeGrupo↵
textoDescritivo↵
↵
Registo de grupo executado.↵
>
```

Exemplo da execução do comando *IG* quando o grupo com o nome *Judo-Caparica-Quarta* já existe no sistema. Neste caso, escreve-se na consola *Grupo ja existente*.

```
> IG Judo-Caparica-Quarta↵
Aula de Judo de Quarta - Escola de Judo da Caparica.↵
↵
Grupo ja existente.↵
>
```

3.7 Comando DG – consulta dados de grupo

Consulta os dados de um dado grupo. Este comando só tem sucesso se o grupo existir. Quando este comando é executado, o nome do grupo e a sua descrição são escritos na consola.

O formato geral do comando é o seguinte:

```
> DG nomeGrupo↵
↵
nomeGrupo↵
textoDescritivo↵
>
```

Exemplo da execução do comando *DG* quando o nome do grupo dado (ex. *grupoX*) não existe.

```
> DU grupoX↵
↵
Inexistencia do grupo referido.
>
```

3.8 Comando RG – remover grupo

Remove um grupo do sistema. Dado um nome de grupo *nomeGrupo*, remove o grupo com este nome. O grupo deve ainda ser removido dos grupos de todos os seus participantes. Este comando só tem sucesso se o grupo com o nome *nomeGrupo* existir.

O formato geral do comando é o seguinte:

```
> RG nomeGrupo↵
↵
Remocao de grupo executada.↵
>
```

Exemplo da execução do comando **RG** quando o grupo com o nome *T1-AED* não existe no sistema. Neste caso, escreve-se na consola *Inexistencia do grupo referido*.

```
> RG T1-AED↵
↵
Inexistencia do grupo referido.↵
>
```

3.9 Comando IP – inserir participante num grupo

Inserir um participante num grupo. Em caso de sucesso, esta operação insere o grupo nos grupos do utilizador e insere o utilizador nos participantes do grupo.

O formato geral do comando é o seguinte:

```
> IP login nomeGrupo↵
↵
Registo de participante executado.↵
>
```

Este comando só tem sucesso se (1) o utilizador com o login *login* existir, se (2) o grupo com o nome *nomeGrupo* existir e se (3) o utilizador com o login *login* ainda não é participante do grupo *nomeGrupo*. A ordem de verificação de erros neste comando é: (1) existência do utilizador, (2) existência do grupo e (3) existência do participante com o login *login* no grupo.

Exemplo da execução do comando **IP** quando o utilizador com o login *bb* não existe no sistema.

```
> IP bb Judo-Caparica-Quarta↵
↵
Inexistencia do utilizador referido.↵
>
```

Exemplo da execução do comando **IP** quando o grupo com o nome *T1-AED* não existe no sistema.

```
> IP aa T1-AED↵
↵
Inexistencia do grupo referido.↵
>
```

Exemplo da execução do comando **IP** quando o utilizador com o login *aa* já é participante do grupo *Judo-Caparica-Quarta*.

```
> IP aa Judo-Caparica-Quarta↵
↵
Existencia de aderencia referida.↵
>
```

Exemplo da execução do comando **IP** quando o utilizador com o login *bb* não existe no sistema e o grupo com o nome *T1-AED* não existe no sistema.

```
> IP bb T1-AED↵
↵
Inexistencia do utilizador referido.↵
>
```

3.10 Comando **RP** – remover participante de um grupo

Remove um participante de um grupo. Em caso de sucesso, esta operação remove o utilizador dado dos participantes do grupo e remove o grupo dos grupos do utilizador.

O formato geral do comando é o seguinte:

```
> RP login nomeGrupo↵
↵
Remocao de aderencia executada.↵
>
```

Este comando só tem sucesso se (1) o utilizador com o login *login* existir, se (2) o grupo com o nome *nomeGrupo* existir e se (3) o utilizador com o login *login* for participante do grupo *nomeGrupo*. A ordem de verificação de erros neste comando é: (1) existência do utilizador, (2) existência do grupo e (3) existência do participante com o login *login* no grupo.

Exemplo da execução do comando **RP** quando o utilizador com o login *bb* não existe no sistema.

```
> RP bb Judo-Caparica-Quarta↵
↵
Inexistencia do utilizador referido.↵
>
```

Exemplo da execução do comando **RP** quando o grupo com o nome *T1-AED* não existe no sistema.

```
> RP aa T1-AED↵
↵
Inexistencia do grupo referido.↵
>
```

Exemplo da execução do comando **RP** quando o utilizador com o login *cc* não é participante do grupo *Judo-Caparica-Quarta*.

```
> IP cc Judo-Caparica-Quarta↵
↵
Inexistencia de aderencia referida.↵
>
```

3.11 Comando **LP** – listagem de participantes

Lista os participantes de um grupo. Listagem, por ordem lexicográfica do login, dos participantes de um dado grupo. Para cada participante, escreve-se na consola o seu login e nome.

O formato geral do comando é o seguinte:

```
> LP nomeGrupo↵
↵
loginP1 nomeP1↵
loginP2 nomeP2↵
...
loginPn nomePn↵
>
```

loginPi e *nomePi* representam o login e nome do *i*-ésimo participante do grupo com nome *nomeGrupo*.

Este comando só tem sucesso se (1) o grupo com nome *nomeGrupo* existir, e se (2) este grupo tem participantes. A ordem de verificação de erros neste comando é: (1) existência do grupo e (2) existência de participantes no grupo.

Quando o grupo não existe no sistema, escreve-se na consola **Inexistencia do grupo referido**. Exemplo da execução do comando **LP** quando o grupo com nome *T1-AED* não existe no sistema.

```
> LP T1-AED↵
↵
Inexistencia do grupo referido.↵
>
```

Quando o grupo não tem participantes, escreve-se na consola **Inexistencia de participantes**. Exemplo da execução do comando **LP** quando o grupo *P2-AED* não tem participantes registados no sistema.

```
> LP P2-AED↵
↵
Inexistencia de participantes.↵
>
```

3.12 Comando **IM** – inserir mensagem

Regista mensagem associada a um dado utilizador. Esta inserção implica também tornar a mensagem visível a todos os contactos do utilizador, assim como a todos os grupos em que o utilizador é participante. Um utilizador pode submeter várias mensagens. Este comando só tem sucesso se o utilizador existir.

O formato geral do comando é o seguinte:

```
> IM login1↵
título↵
textoMensagem↵
URL↵
↵
Registo de mensagem executado.↵
>
```

Exemplo da execução do comando `IM` quando o utilizador com o login `bb` não existe no sistema.

```
> IM bb↵
Medidas covid-19 na FCT↵
Este site tem informacao que pode interessar.↵
www.fct.unl.pt/covid-19↵
↵
Inexistencia do utilizador referido.↵
>
```

3.13 Comando **LMC** – listar mensagens de contacto

Lista as mensagens de um contacto. Pedido de listagem das mensagens de um utilizador, por ordem de inserção, da mais recente para a mais antiga. O pedido é submetido por um dos seus contactos.

O formato geral do comando é o seguinte:

```
> LMC login1 login2↵
↵
títuloMn↵
textoMn↵
urlMn↵
↵
...
títuloM2↵
textoM2↵
urlM2↵
↵
títuloM1↵
textoM1↵
urlM1↵
↵
>
```

$tituloMi$, $textoMi$ e $urlMi$ representam o título, texto e URL da i -ésima mensagem do utilizador com login `login1`. Estas mensagens podem ter sido escritas pelo próprio utilizador `login1` ou podem ser mensagens que este utilizador recebeu (**durante toda a execução do sistema**). O pedido é feito pelo utilizador com login `login2` e os dois utilizadores são contacto um do outro.

Note que as mensagens inseridas pelo utilizador `login2` só começam a ser enviadas ao utilizador com login `login1` quando estes utilizadores registam o contacto. As mensagens inseridas pelo utilizador `login2` depois da remoção do contacto, não são enviadas ao utilizador `login1`. Por exemplo, suponha que o utilizador `login2` não é contacto do utilizador `login1` e que este último

não enviou nem recebeu mensagens até ao momento. Suponha ainda que o utilizador *login2* insere as mensagens m_1 , m_2 e m_3 . Depois disso um dos utilizadores regista contacto com o outro. De seguida, o utilizador *login2* insere as mensagens m_4 , m_5 e m_6 . Como resultado, estas mensagens são enviadas ao utilizador *login1*. O comando `LMC login1 login2` teria como resultado mostrar a informação das mensagens m_6 , m_5 e m_4 por esta ordem.

Assuma que de seguida o utilizador *login2* remove o contacto com o utilizador *login1* e insere a mensagem m_7 . Esta mensagem já não é enviada ao utilizador *login1*. Como nesta altura os dois utilizadores já não são contacto um do outro, o comando `LMC login1 login2` já não lista as mensagens do utilizador *login1*.

O comando `LMC login1 login1` resulta na listagem de todas as mensagens escritas e recebidas pelo utilizador *login1*. No exemplo anterior, este comando resultaria na listagem das mensagens m_6 , m_5 e m_4 por esta ordem.

Este comando só tem sucesso se (1) os dois utilizadores existirem, se (2) o contacto entre os dois utilizadores foi registado no sistema e se (3) houver mensagens para listar. A ordem de verificação de erros neste comando é: (1) existência dos utilizadores, (2) existência do contacto entre os dois utilizadores e (3) há mensagens para listar.

Quando um dos utilizadores, ou ambos, não existe no sistema, escreve-se na consola `Inexistencia do utilizador referido.` Exemplo da execução do comando `LMC` quando o utilizador com o login *bb* não existe no sistema.

```
> LMC aa bb↵
↵
Inexistencia do utilizador referido.↵
>
```

Exemplo da execução do comando `LMC` quando o contacto entre os utilizadores com os logins *aa* e *cc* não foi registado, ou já foi removido.

```
> LMC aa cc↵
↵
Inexistencia do contacto referido.↵
>
```

Exemplo da execução do comando `LMC` quando o contacto entre os utilizadores com os logins *aa* e *dd* foi registado mas *aa* não tem mensagens.

```
> LMC aa dd↵
↵
Contacto nao tem mensagens.↵
>
```

Exemplo

Vejamos um exemplo. Assuma que os utilizadores com os valores de login *aa*, *cc* e *dd* foram introduzidos como utilizadores, que não são contactos uns dos outros e que até ao momento não foram introduzidas quaisquer mensagens no sistema. Assuma também que a próxima listagem de comandos foi introduzida:

```

> IM dd↵
título D0↵
texto D0↵
urlD0↵
↵
Registo de mensagem executado.↵
> IC aa dd↵
↵
Registo de contacto executado.↵

```

```

> IM dd↵
título D1↵
texto D1↵
urlD1↵
↵
Registo de mensagem executado.↵
> IM dd↵
título D2↵
texto D2↵
urlD2↵
↵
Registo de mensagem executado.↵
> IC aa cc↵
↵
Registo de contacto executado.↵
> IM cc↵
título C1↵
texto C1↵
urlC1↵
↵
Registo de mensagem executado.↵

```

Como os utilizadores *aa* e *dd* são contactos, o utilizador *dd* pode pedir para ver as notificações que o *aa* recebeu. Para isso usa o comando **LMC aa dd**, que mostra todas as mensagens escritas e recebidas pelo utilizador *aa*. Verifica-se o mesmo resultado com o comando **LMC aa cc**, pois *cc* também é contacto de *aa*. O utilizador *aa* também pode ver as notificações que os seus contactos enviaram e receberam. Por exemplo, o comando **LMC dd aa** lista todas as notificações escritas e recebidas por *dd*.

```

> LMC aa dd↵
↵
TITULO C1↵
TEXTO C1↵
URLC1↵
↵
TITULO D2↵
TEXTO D2↵
URLD2↵
↵
TITULO D1↵
TEXTO D1↵
URLD1↵
↵

```

```

> LMC aa cc↵
↵
TITULO C1↵
TEXTO C1↵
URLC1↵
↵
TITULO D2↵
TEXTO D2↵
URLD2↵
↵
TITULO D1↵
TEXTO D1↵
URLD1↵
↵

```

```

> LMC dd aa↵
↵
TITULO D2↵
TEXTO D2↵
URLD2↵
↵
TITULO D1↵
TEXTO D1↵
URLD1↵
↵
TITULO D0↵
TEXTO D0↵
URLD0↵
↵

```

Suponha que de seguida os utilizadores *aa* e *dd* deixam de ser contactos. Neste caso, deixam de poder ver as notificações do outro.

```
> RC aa dd↵
↵
Remocao de contacto executada.↵
> LMC aa dd↵
↵
Inexistencia do contacto referido.↵
> LMC dd aa↵
↵
Inexistencia do contacto referido.↵
```

Os utilizadores *aa* e *cc* continuam a conseguir ver as notificações do outro porque ainda são contactos. O resultado dos pedidos de leitura de notificações *LMC cc aa* e *LMC aa cc* depois de *aa* introduzir uma mensagem estão indicados em baixo.

```
> IM aa↵
titulo A1↵
texto A1↵
urlA1↵
↵
Registo de mensagem executado.↵
```

```
> LMC aa cc↵
↵
TITULO A1↵
TEXTTO A1↵
URLA1↵
↵
TITULO C1↵
TEXTTO C1↵
URLC1↵
↵
TITULO D2↵
TEXTTO D2↵
URLD2↵
↵
TITULO D1↵
TEXTTO D1↵
URLD1↵
↵
```

```
> LMC cc aa↵
↵
TITULO A1↵
TEXTTO A1↵
URLA1↵
↵
TITULO C1↵
TEXTTO C1↵
URLC1↵
↵
```


Se o contacto entre *aa* e *dd* voltar a ser registado, estes utilizadores voltam a ter permissão de leitura das notificações um do outro.

```
> IC dd aa↵
```

```
↵
```

```
Registo de contacto executado.↵
```

```
> LMC dd aa↵
```

```
↵
```

```
TITULO D2↵
```

```
TEXT0 D2↵
```

```
URLD2↵
```

```
↵
```

```
TITULO D1↵
```

```
TEXT0 D1↵
```

```
URLD1↵
```

```
↵
```

```
TITULO D0↵
```

```
TEXT0 D0↵
```

```
URLD0↵
```

```
↵
```

```
> LMC aa dd↵
```

```
↵
```

```
TITULO A1↵
```

```
TEXT0 A1↵
```

```
URLA1↵
```

```
↵
```

```
TITULO C1↵
```

```
TEXT0 C1↵
```

```
URLC1↵
```

```
↵
```

```
TITULO D2↵
```

```
TEXT0 D2↵
```

```
URLD2↵
```

```
↵
```

```
TITULO D1↵
```

```
TEXT0 D1↵
```

```
URLD1↵
```

```
↵
```

Qualquer utilizador pode ver as suas notificações em qualquer momento.

```
> LMC aa aa↵
```

```
↵
```

```
TITULO A1↵
```

```
TEXT0 A1↵
```

```
URLA1↵
```

```
↵
```

```
TITULO C1↵
```

```
TEXT0 C1↵
```

```
URLC1↵
```

```
↵
```

```
TITULO D2↵
```

```
TEXT0 D2↵
```

```
URLD2↵
```

```
↵
```

```
TITULO D1↵
```

```
TEXT0 D1↵
```

```
URLD1↵
```

```
↵
```

3.14 Comando **LMG** – listar mensagens de grupo

Lista as mensagens de um grupo. Pedido de listagem das mensagens de um grupo, por ordem de inserção, da mais recente para a mais antiga. O pedido é submetido por um dos seus participantes e tem como resultado a listagem de todas as mensagens recebidas pelo grupo (**durante toda a execução do sistema**). O formato geral do comando é o seguinte:

```
> LMG nomeGrupo login↵
↵
tituloMn↵
textoMn↵
urlMn↵
↵
...
tituloM2↵
textoM2↵
urlM2↵
↵
tituloM1↵
textoM1↵
urlM1↵
↵
>
```

tituloMi, *textoMi* e *urlMi* representam o título, texto e URL da *i*-ésima mensagem recebida pelo grupo com nome *nomeGrupo*. O pedido é feito pelo utilizador com login *login* que é participante deste grupo.

Note que as mensagens inseridas pelo utilizador com login *login* só começam a ser enviadas ao grupo *nomeGrupo* quando este utilizador se torna participante deste grupo. As mensagens inseridas pelo utilizador depois deste sair do grupo, não são enviadas ao grupo. Por exemplo, suponha que o utilizador com o login *loginU*, não é participante do grupo com nome *nomeG* e que este último não recebeu nenhuma mensagem até ao momento. Suponha ainda que este utilizador insere as mensagens m_1 , m_2 e m_3 . Depois disso torna-se participante deste grupo. De seguida insere as mensagens m_4 , m_5 e m_6 . Como resultado, estas mensagens são enviadas ao grupo. O comando **LMG nomeG loginU** teria como resultado mostrar a informação das mensagens m_6 , m_5 e m_4 por esta ordem.

Assuma ainda que de seguida o utilizador *loginU* sai do grupo e insere a mensagem m_7 . Esta mensagem já não é enviada ao grupo *nomeG*. Como nesta altura o utilizador já não é participante deste grupo, o comando **LMG nomeG loginU** já não lista as mensagens do grupo *grupoG*. No entanto, se um outro participante do grupo pedir uma listagem das notificações do grupo, teria como resultado a listagem das mensagens m_6 , m_5 e m_4 por esta ordem.

Este comando só tem sucesso se (1) o grupo com o nome *nomeGrupo* existir, se (1) o utilizador com o login *login* existir, se (3) o utilizador com o login *login* for participante do grupo *nomeGrupo* e se (4) houver mensagens para listar. A ordem de verificação de erros neste comando é: (1) existência do grupo, (2) existência do utilizador, (3) existência do participante com o login *login* no grupo e (4) há mensagens para listar.

Exemplo da execução do comando **LMG** quando o grupo com o nome *T1-AED* não existe no sistema.

```
> LMG T1-AED cc↵
↵
Inexistencia do grupo referido.↵
>
```

Exemplo da execução do comando **LMG** quando o utilizador com o login *bb* não existe no sistema.

```
> LMG P2-AED bb↵
↵
Inexistencia do utilizador referido.↵
>
```

Exemplo da execução do comando **LMG** quando o utilizador com o login *cc* não é participante do grupo *P2-AED*.

```
> LMG P2-AED cc↵
↵
Inexistencia do participante referido.↵
>
```

Exemplo da execução do comando **LMG** quando o grupo com o nome *P2-AED* não tem mensagens. O utilizador *aa* é participante deste grupo.

```
> LMG P2-AED aa↵
↵
Grupo nao tem mensagens.↵
>
```

3.15 Comando **FIM** – terminar

Termina a execução do programa. O comando não tem parâmetros e tem sempre sucesso. O formato do comando é o seguinte:

```
> FIM↵
↵
Obrigado. Ate a proxima.↵
>
```

4 Fases do trabalho e avaliação

O trabalho é obrigatório para todos os alunos que não têm frequência e dá frequência. A avaliação incidirá sobre todos os aspectos: concepção, qualidade e eficiência da solução, modularidade, estrutura e documentação do código, qualidade do relatório final, etc. Se uma das fases não for entregue dentro do prazo definido, a nota associada a essa fase será 0 (zero).

As diferenças entre a fase 1 e a fase 2 são as seguintes:

- Na **Fase 1** os alunos devem fazer o desenho da solução. Este desenho consiste no modelo dos Tipos Abstratos de Dados (TAD) deste problema. O modelo deve incluir os TADs de domínio e os TADs genéricos (ver slide 12 da Aula-1).

Os alunos devem definir as interfaces e classes que irão implementar cada TAD. Nesta fase, **as colecções devem ser implementadas com listas ligadas ou vectores**.

A implementação desta fase terá que ser submetida no Mooshak (secção 6).

- Na **Fase 2** os alunos devem melhorar as complexidades temporal e espacial da sua solução. Para isso, devem substituir os TADs genéricos usados na fase 1 (assim como as interfaces e classes que os implementam) por outros mais eficientes.

A implementação desta fase terá que ser submetida no Mooshak e o relatório deve ser entregue no Moodle. O modelo dos Tipos Abstratos de Dados final (ou seja, o modelo usado na fase 2) deve ser entregue no relatório final.

Em ambas as fases, os alunos podem (e devem) mostrar o modelo dos Tipos Abstratos de Dados aos docentes durante a aulas práticas para receberem feedback sobre a sua solução.

Em qualquer uma das fases, os alunos podem optar por uma das seguintes versões:

- Na **Versão I**, avaliada entre 0 e 20 valores, não é permitido fazer uso do pacote *java.util*, à excepção da classe *Scanner*.
- Na **Versão J** é permitido usar algumas interfaces e classes do pacote *java.util*. Os trabalhos que só usam interfaces e classes do pacote *java.util* e não implementam nenhuma das interfaces e classes do pacote *dataStructures*, são avaliados entre 0 e 11 valores. Haverá uma bonificação por cada classe do pacote *dataStructures* que for implementada (sendo a nota máxima correspondente a 20 valores).

5 Submissão

A fase 1 e a fase 2 envolvem a entrega de um programa implementado em Java. O programa deve ser entregue nos concursos do Mooshak para o efeito. Só serão avaliados os trabalhos que passam os Testes de Avaliação do Mooshak. Adicionalmente, na última fase deverá ainda ser entregue no Moodle um relatório final do trabalho.

As datas de entrega são:

- **30 de outubro** – submissão da primeira fase do trabalho, no Mooshak até às 17h.
- **4 de dezembro** – submissão da segunda fase do trabalho no Mooshak e do relatório final no Moodle até às 17h.

Cada grupo de 2 alunos (equipa de trabalho) deve registar-se nesses concursos. O nome do utilizador deve ser o seu **número de aluno_número de aluno** (o primeiro número a colocar é o menor) no grupo correspondente ao seu turno (por exemplo, os alunos nº 3435 do turno P1 e nº 3434 do turno P5 devem ter como nome utilizador no Mooshak **3434_3435** no grupo **P1**). Só serão aceites como entregas para avaliação, os programas cujo utilizador no concurso do Mooshak siga estas regras.

A entrega no Mooshak, é constituída por um zip contendo o código fonte devidamente comentado, tal como descrito na secção 6. O nome e número dos alunos que compõem o grupo deverá ser inserido em todos os ficheiros entregues.

Para os programas e relatório serem avaliados, os alunos devem ter cumprido o código de ética do DI, e ter, no final do prazo, uma entrega dum programa no concurso do Mooshak associado a essa entrega. Pode submeter o seu código mais que uma vez. Só a última submissão será avaliada.

6 Requisitos do Mooshak

Para submeter o trabalho ao Mooshak, é necessário que o programa fonte respeite as quatro regras seguintes:

- O código fonte completo (ficheiros *.java) tem de ser guardado num único arquivo ZIP.
- A classe principal tem de se chamar *Main* e tem de estar na raiz do arquivo (i.e., tem de pertencer ao pacote principal (*default*)).
- As pastas correspondentes aos pacotes do trabalho têm de estar na raiz do arquivo.
- O programa só pode criar ficheiros na directoria corrente.

Para evitar problemas causados pela dimensão do ficheiro submetido, somente o código fonte (ficheiros *.java) deve ser guardado no arquivo ZIP.

7 Testes e discussão

O trabalho terá de satisfazer todos os requisitos especificados na secção 3. Essa verificação será efetuada automaticamente pelo sistema Mooshak, com os Testes de Avaliação.

Em cada fase:

- se o programa não passar todos os Testes de Avaliação, os seus autores obterão a nota de 0 (zero) na fase em questão;
- se o programa passar todos os Testes de Avaliação da fase, os docentes farão uma avaliação preliminar da fase, que deverá ser confirmada no final do semestre, numa discussão final.

No final do semestre, todos os grupos em posição de obter frequência serão submetidos a uma discussão do trabalho, para verificação de autoria. Nesta discussão, os membros do grupo poderão ser questionados sobre **qualquer das classes e interfaces submetidas ao Mooshak em qualquer das fases de submissão**.

Se se detetar:

- que um trabalho não foi realizado apenas pelos alunos que o assinaram;
- que um aluno assinou um trabalho que não realizou; ou
- que a distribuição das tarefas pelos membros do grupo não foi equilibrada,

esse trabalho será anulado e nenhum dos elementos do(s) grupo(s) envolvido(s) obterá frequência. Se um aluno faltar à discussão do trabalho que assinou, não obterá frequência, reprovando à disciplina.